

Lesson 12

by Lorraine Gaudio

Lesson generated on November 09, 2025



BOISE STATE UNIVERSITY

Contents

1	☒ Welcome Back to R!	3
2	Distribution of Sums	4
3	Warm☒Up	7
4	Simulating	8
4.1	☒ Coin Flips	8
4.2	☒ Dice Rolls	9
5	☒ Histograms	10
6	☒☒ Two Dice	13
7	☒ Symmetric Data	14
8	☒ Repeating a Simulation	17
8.1	Normal Data with <code>replicate()</code>	18
9	☒ Fair-Dice Check	22
9.1	Biased Die	22
9.2	Fair vs. Unfair Dice	23
10	☒ Practice Space	25
11	☒ Assignment	26
11.1	Dataset	26
11.2	Objective	28
11.3	Lognormal Random Numbers	28
11.4	Task 1	29
11.5	Task 2	29
11.6	Task 3	30
11.7	Task 4	30
11.8	Task 5	31
11.9	Task 6	31
11.10	Task 7	31
11.11	Task 8	32
11.12	Task 9	33
11.13	Task 10	33
11.14	Reflections	34

12	Save and Upload	35
13	Today you practiced:	36

1. ☒ Welcome Back to R!

We shift ☐ gears from the `dyplr` functions to using R's built-in tools to visualize simulations. Simulation lets you create synthetic data, test ideas quickly, and understand probability without a math degree. We will learn three key functions: `sample()`, `rnorm()`, and `replicate()`. While we are at it, we'll learn how to use base R to visualize the results using `hist()`.

☐ No extra packages are needed today. We'll be using built-in stats functions.

To begin Lesson 12, follow these steps:

1. Open your course project for RStudio
2. Create a new file. From the file types we have used so far, pick which file type you want to use. (File > New File > ???).
3. Type in the code provided in this document as you follow along with the video. Pause the video at anytime to answer assignment questions, dig deeper or add memo notes. In the same file, complete the assignment at the end of this document. Use memos in your assignment portion to explain your thinking.

Lesson Overview

By the end of Lesson 12 you will be able to:

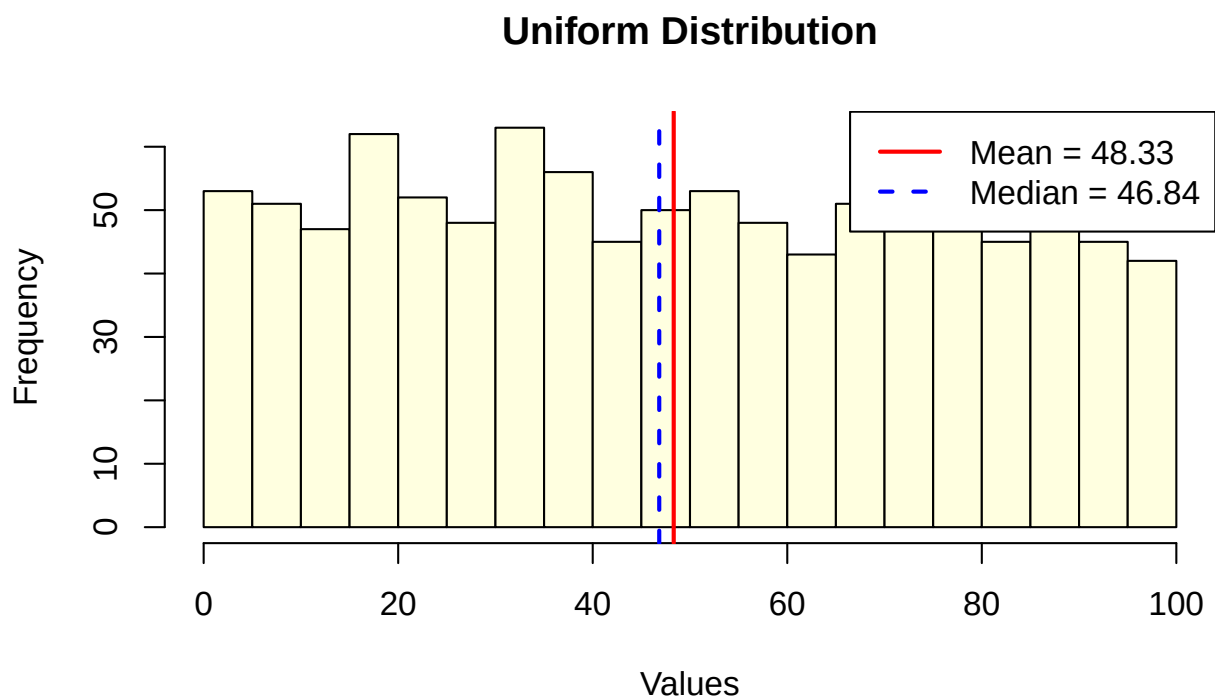
1. ☐ Remember – Name `sample()`, `rnorm()`, and `replicate()`.
2. ☐ Understand – Explain when to use replacement and probability weights in `sample()`.
3. ☐ Apply – Simulate coins, dice, and normal data with code
4. ☐ Analyze – Visualize simulated results with `hist()` and interpret shapes.
5. ☐ Evaluate – Judge whether simulated outcomes match an expected distribution (fair vs. biased).

Keep these goals in mind as you move through each section.

2. Distribution of Sums

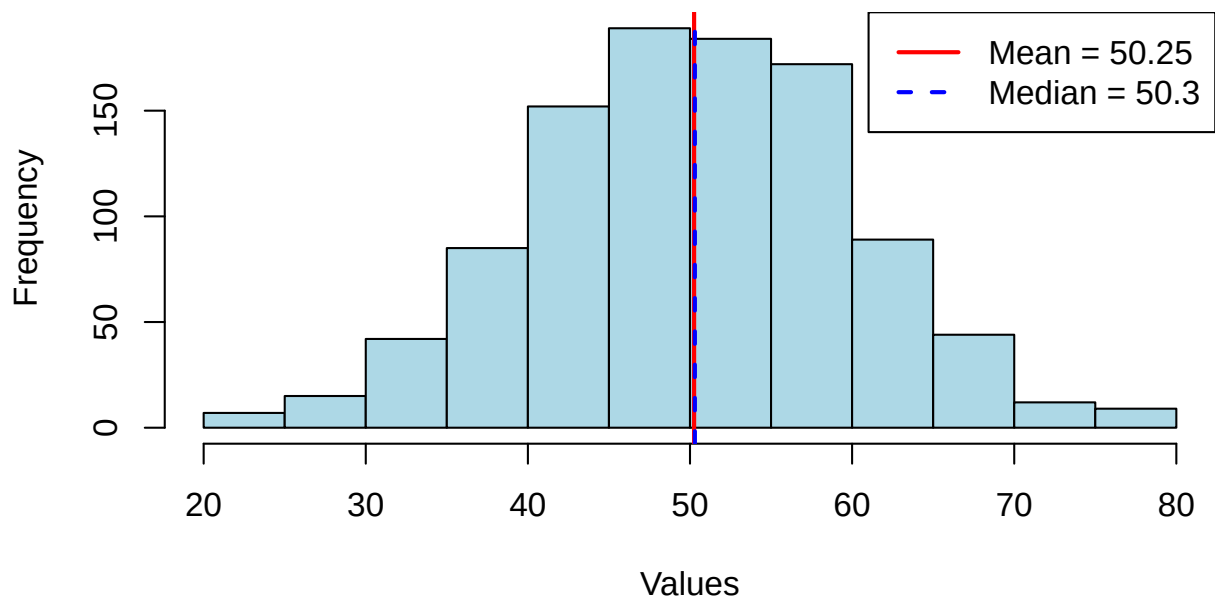
□ Identify the “Shape of Data” (or “Distribution of data”) is *optional reading*, but may help you answer questions in this lesson.

1. **Uniform:** All bars have approximately the same height, indicating that all values are equally likely.



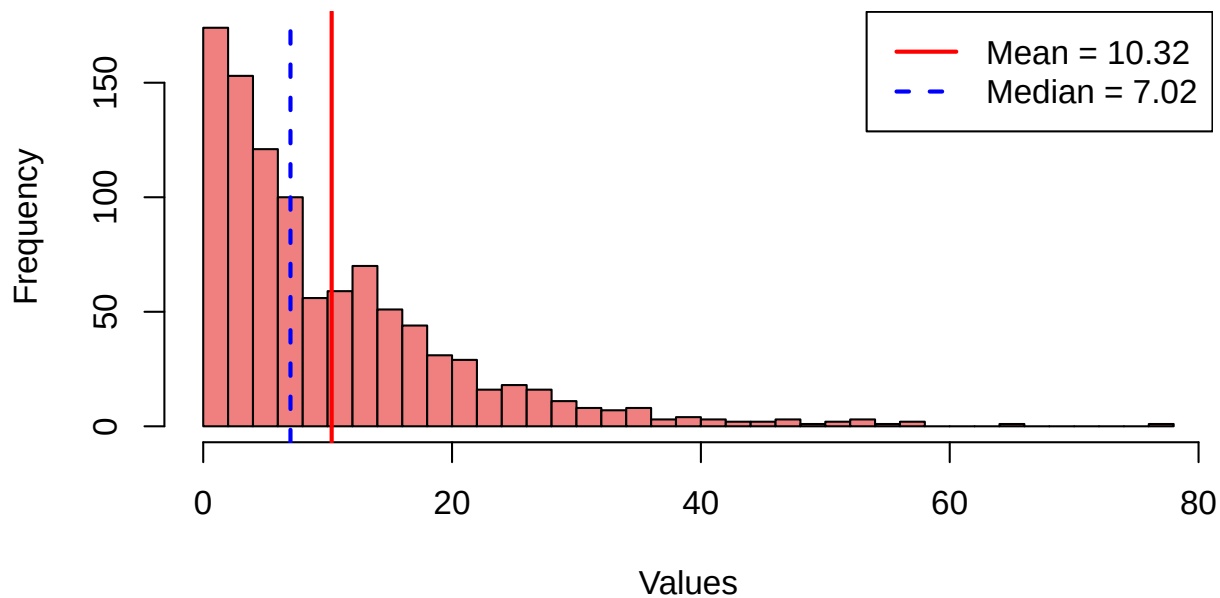
2. **Symmetric:** This bell-shaped (normal) distribution is the left and right sides of the histogram are nearly mirror images of each other. Mean and median are approximately equal. $Median \approx Mean$.

Symmetric Distribution



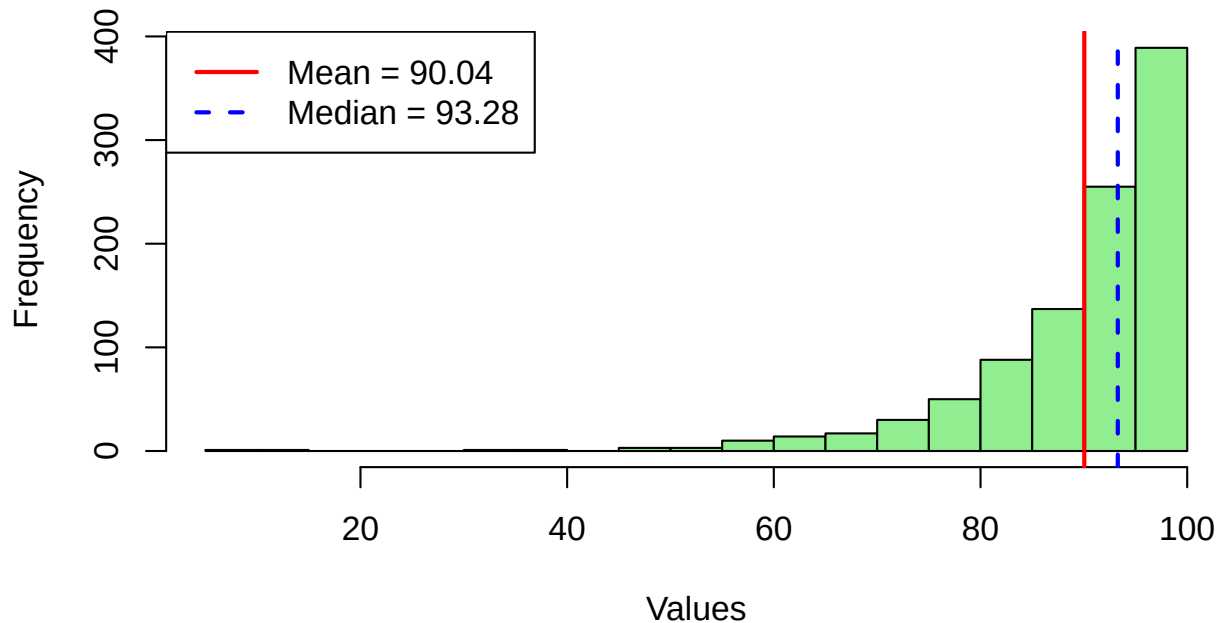
3. **Skewed Right:** The right *tail* is longer than the left tail, indicating that there are a few high values that pull the mean to the right. *Positive skewness*. The mean is more right than the median due to extreme values. $Median < Mean$.

Right Skewed Distribution



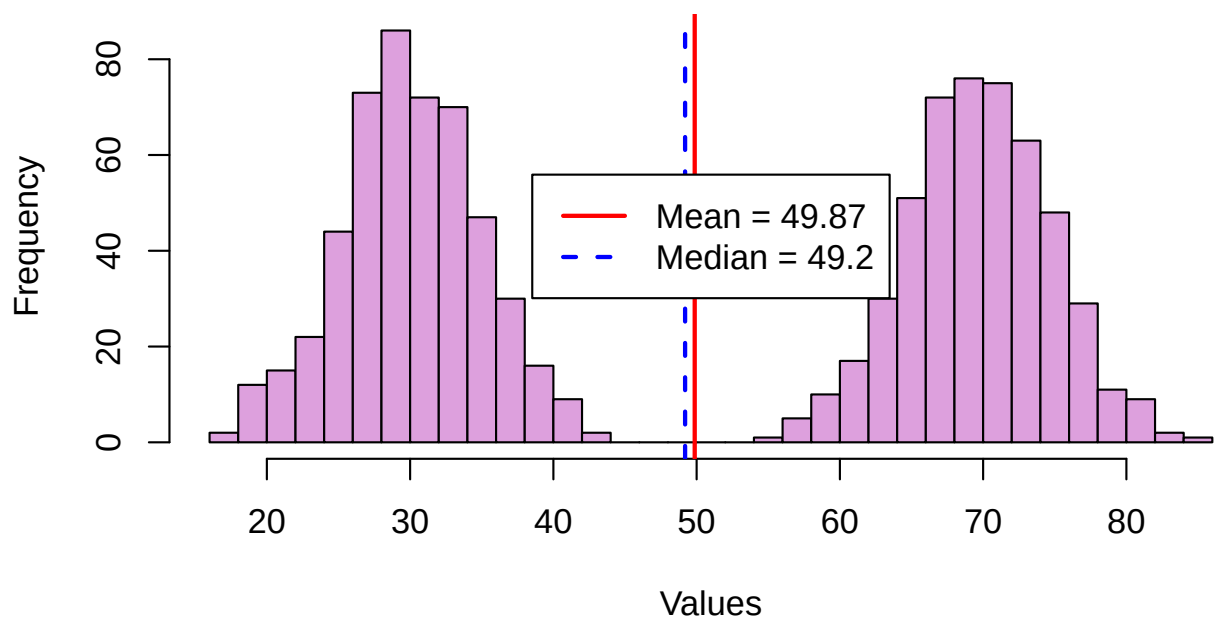
4. **Skewed Left:** The left *tail* is longer than the right tail, indicating that there are a few low values that pull the mean to the left. *Negative skewness*. $Median > Mean$.

Left Skewed Distribution



5. **Bimodal (Split Distribution):** The distribution has two distinct peaks or modes. This suggests that the data might come from two different groups or processes. The mean and median may fall between the peaks, providing misleading summary statistics. Bimodal distributions often indicate hidden subpopulations.

Bimodal Distribution



3. WarmUp

A data analyst uses `set.seed()` to ensure that random processes like sampling or simulation produce the same results each time the code is run (from top to bottom). This reproducibility is crucial for debugging, collaboration, and verifying results, as it guarantees consistency across analyses and users.

```
# □ Test  
set.seed(2025) # □ for reproducibility
```

If you re-run code more than once within your document, your values will not be the same. It is only the same when you run the code from top to bottom in one session.

4. Simulating

4.1 🎲 Coin Flips

□ The GOAL: Flip □ 1,000 fair coins (0 = heads, 1 = tails) and count tails.

Terminology: **Sample Space**: “ S ” A collection (or set) of all possible outcomes in a random experiment.

Set Notation: $S = \{\text{Head}, \text{Tail}\}$

1. □ Create the data through a simulation

```
# □ Test
# sample(x, size, replace = FALSE, prob = NULL)
coins <- sample(x = 0:1, size = 1000, replace = TRUE)
```

□ Comment (example): $x = 0:1$ means that we are sampling from either 0 or 1. As the researcher, I decide that 0 = heads and 1 = tails. We set $\text{size} = 1000$ to flip the coin 1,000 times. We set $\text{replace} = \text{TRUE}$ because we “put the coin back” each flip.

```
# □ Verify
unique(coins) # values in our data set
length(coins) # total number of flips
typeof(coins) # type of object
```

□ Comment (example): Coins contains 0 and 1 values where 1 represents tails of a coin flip. There are 1000 values in the integer vector.

□ Prediction: How many tails are in our data set?

□ Use sum to count the number of tails because 1 = tails

```
# □ Test
sum(coins)
```

□ Comment (your turn): How many tails are in our data set? Why does $\text{sum}(\text{coins})$ count the tails?

□ Explore and Play: Can you make a new data set where heads = 1? Try $1 - \text{coins}$ and sum . Why would this work?

□ Break Things! What happens if $\text{replace} = \text{FALSE}$ keeping $\text{size} = 1000$? Memo: paste the error message and 1 sentence on why it occurs.

4.2 🎲 Dice Rolls

□ The GOAL: Simulate 10,000 rolls of a fair 6-sided die and plot the results.

Set Notation: $S = \{1, 2, 3, 4, 5, 6\}$

□ Prediction: How many times do you expect each face to appear? Will the values be exactly equal? Have a normal distribution? Something else? Type your answer before working ahead. *Keep whatever you typed, even if you are wrong. That's the integrity of science!*

```
# □ Test  
dice <- sample(1:6, 10000, replace = TRUE)
```

To investigate our hypothesis, we'll visualize the results with a histogram using base R.

5. 📊 Histograms

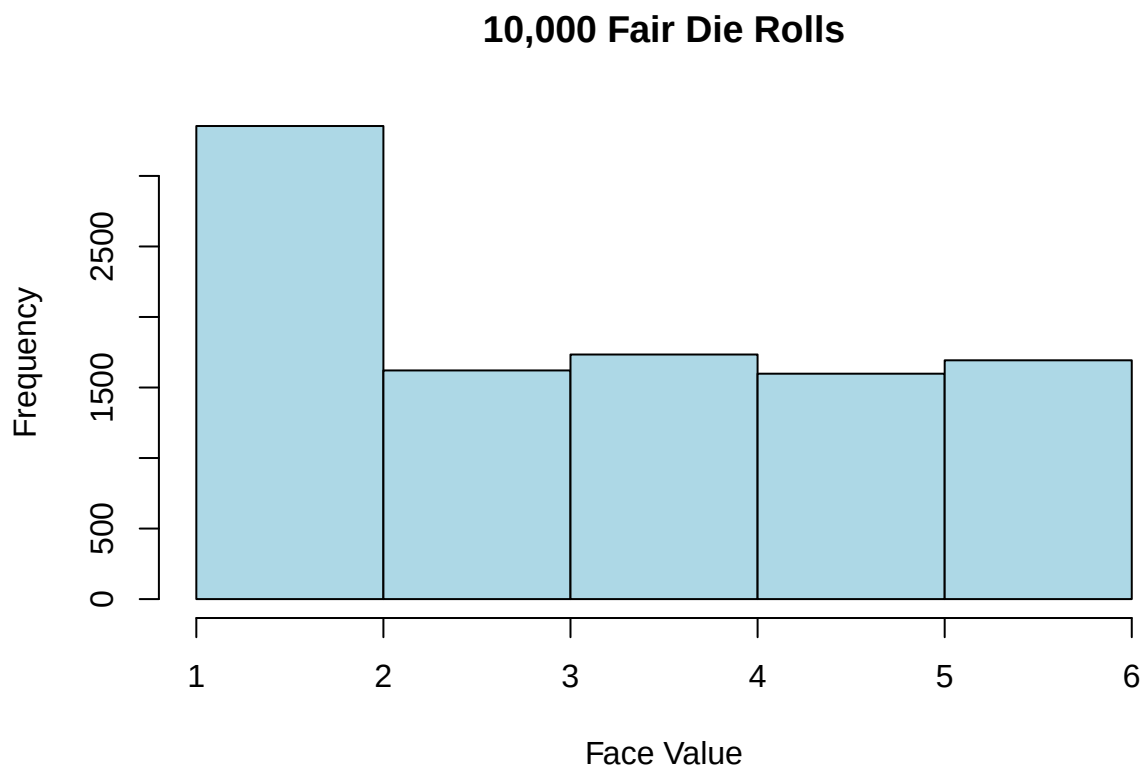
- Use `hist()` to create a histogram of the dice rolls, showing the frequency of each face value.

```
?hist
```

- The SYNTAX: `hist(x, breaks, col, main, xlab, ylab)`

- `x` is the data to plot
- `breaks` controls the number of bins (or bars)
- `col` sets the color of the bars
- `main` is the title of the plot
- `xlab` and `ylab` label the x- and y-axes

```
# □ Verify
hist(dice,                # The data to plot
     breaks = 6,
     col = "lightblue",   # Color of the bars
     main = "10,000 Fair Die Rolls", # Title of the plot
     xlab = "Face Value") # Label for the x-axis
```

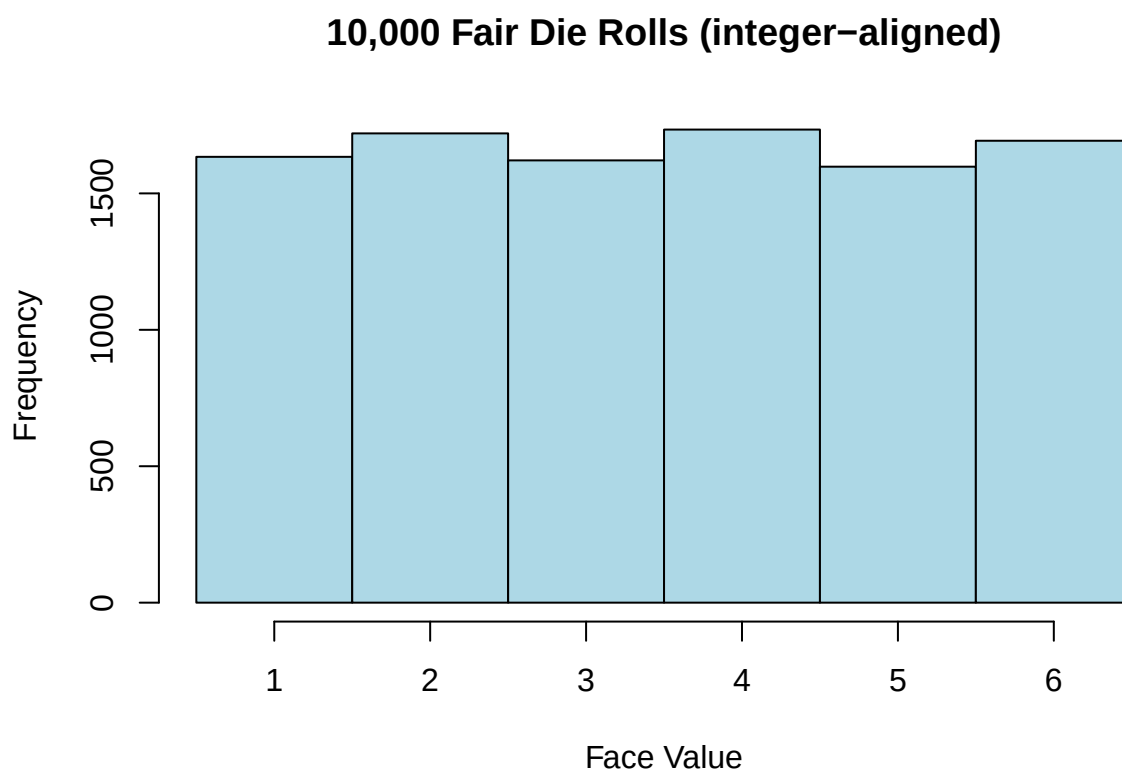


- Comment (example): The y-axis label is automatically set to “Frequency” by default.
- NOTICE: Bars should be roughly equal height, forming a uniform distribution. That isn’t what we see.

When we specify `breaks = 6`, base R doesn’t create bins that align perfectly with our integer values 1-6. The bins slice the integers unevenly, leading to misleading bar heights.

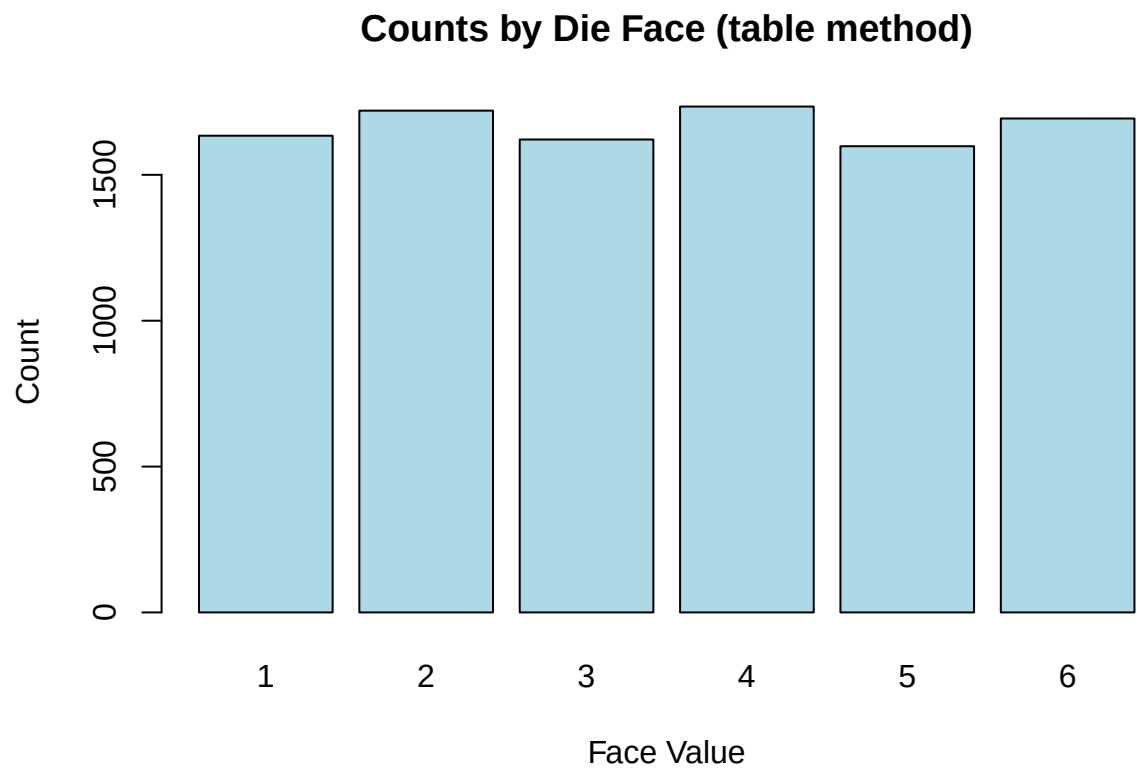
- The GOAL: Fix the breaks to align with die face values.

```
# □ Test
hist(dice,
  breaks = seq(0.5, 6.5, by = 1), # centers bars on 1:6
  col = "lightblue",
  main = "10,000 Fair Die Rolls (integer-aligned)",
  xlab = "Face Value")
```



- Comment (example): The bars look more even now.

```
# Alternative display for discrete data
barplot(table(dice),
  main = "Counts by Die Face (table method)",
  col = "lightblue",
  xlab = "Face Value",
  ylab = "Count")
```



Comment (example): The `table()` function provides an alternative way to display the data accurately. □

6. 🎲 Two Dice

□ Prediction: What histogram shape do you expect when summing the face value of two dice? Will the values be exactly equal? Have a normal distribution? Something else? Type your answer before working ahead. *Keep whatever you typed, even if you are wrong.*

□ The GOAL: Roll two dice 10,000 times, add the results, plot.

```
# □ Test
sum_two <- sample(1:6, 10000, TRUE) + sample(1:6, 10000, TRUE)

# □ Investigate
hist(sum_two,
     breaks = seq(1.5, 12.5, by = 1), # Creates bins centered on integers 2-12
     col = "salmon",
     main = "Sum of Two Dice (10,000 trials)",
     xlab = "Total")
```

□ Comment (example): The histogram shows a bell-shaped (normal) distribution, peaking around 7. This shape arises because there are more combinations to achieve sums near the middle (like 6, 7, and 8) compared to extreme sums (like 2 or 12).

□ Explore and Play: We used the column “salmon” to create the figure. Test out other col options. Which are your favorite?

```
colors() # Lists all color names in R
```

7. ☒ Symmetric Data

The functions `rnorm()` generates a vector of random numbers that form a normal (bell-shaped) distribution.

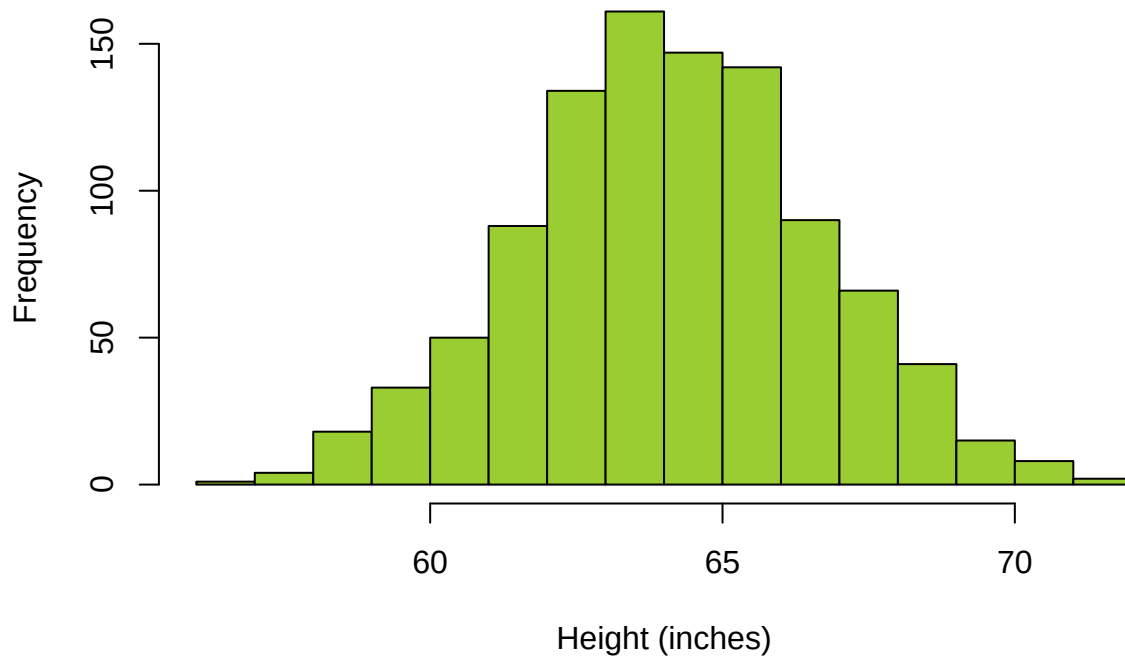
```
?rnorm # Check the help file for rnorm()
```

- ☐ The SYNTAX: `rnorm(n, mean = 0, sd = 1)`
 - `n` is the number of random numbers to generate.
 - `mean` is the average value of the distribution.
 - `sd` is the standard deviation, controlling the spread of the distribution.
- ☐ The GOAL: Simulate 1,000 adult female heights $\sim N(64, 2.5^2)$ and inspect.

```
# ☐ Test  
heights <- rnorm(n = 1000, mean = 64, sd = 2.5)
```

```
# ☐ Verify  
hist(heights,  
      col = "yellowgreen",  
      main = "Simulated Female Heights",  
      xlab = "Height (inches)")
```

Simulated Female Heights



The breaks = are unnecessary here because `hist()` automatically chooses appropriate bins for continuous data.

```
# Verify  
"Mean = "
```

```
## [1] "Mean = "
```

```
mean(heights)
```

```
## [1] 64.11653
```

```
"SD = "
```

```
## [1] "SD = "
```

```
sd(heights)
```

```
## [1] 2.508912
```

☐ Comment (example): The histogram shows a bell-shaped distribution of heights, centered around the mean of 64 inches. The spread of the data is consistent with the specified standard deviation of 2.5 inches.

Sample vs. Population Parameters: You might wonder, “Why aren’t the sample mean and standard deviation exactly 64 and 2.5?” This is because these are random samples from a normal distribution with those parameters. Each time we run the simulation, we get slightly different values due to sampling variability.

In summary, the function `rnorm()` generates random numbers from a normal distribution and is used for modeling normally distributed data using the parameters: number of values, mean, and standard deviation. This will only generate symmetric data.

8. ☒ Repeating a Simulation

The function `replicate()` repeats an expression multiple times and collects the results. This control structure is like a loop is used to repeat sample estimate probabilities (simulations), resample data to estimate variability or confidence intervals (bootstrapping), model repeated trials of an experiment, generate synthetic data and more. This function is flexible as it can wrap any code, not just random generation.

That said, Monte Carlo simulations are the classic use case for `replicate()`. These simulations model are used to understand and manage risk and uncertainty using probabilities.

```
?replicate # Check the help file for replicate()
```

☐ The SYNTAX: `replicate(n, expr, simplify = "array")`

- `n` is the number of replications.
- `expr` is the expression (a language object, usually a call) to evaluate repeatedly.
- `simplify` indicates whether the result should be simplified to a vector, matrix or array if possible. The default is "array".

☐ The GOAL: How variable is the number of tails in 5,000 flips? Repeat 100 times.

☐☐☐ ☐ That's a lot of coins tosses!

```
# ☐ Test  
tails_100 <- replicate(n = 100, expr = sum(sample(0:1, 5000, TRUE)))
```

The `replicate()` function:

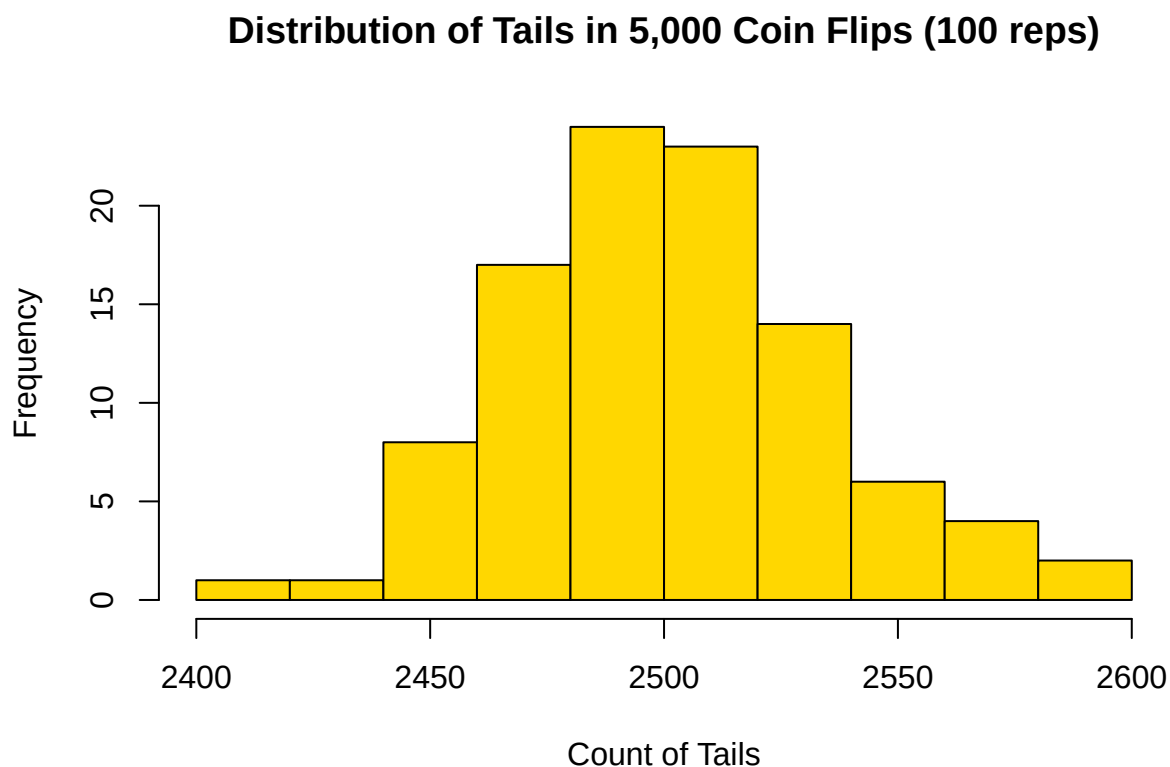
1. Evaluates the entire expression 100 times (because `n = 100`)
2. Each evaluation is independent (gets its own random sampling)
3. Store the 100 results in a vector called `tails_100`

```
# ☐ Verify  
summary(tails_100)
```

```
##   Min. 1st Qu.  Median    Mean 3rd Qu.   Max.     
##  2412   2477   2499   2502   2525   2582
```

□ Comment (example): Sampling variability show that the number of tails varies across these simulations, with a minimum of 2,412 tails and a maximum of 2,582 tails. The mean number of tails is approximately 2,499, which is close to the expected value of 2,500 for a fair coin.

```
# □ Verify  
hist(tails_100,  
  col = "gold",  
  main = "Distribution of Tails in 5,000 Coin Flips (100 reps)",  
  xlab = "Count of Tails")
```



□ Comment (example): The spread shows sampling variability around the theoretical 2,500.

8.1 Normal Data with **replicate()**

□ Scenario. Pretend that the university is considering making specifications grading standard practice. The provost's office says that if thirty percent or more of students oppose this, it would trigger a halt to the proposal. We want to know the risk of the proposal being halted.

We run a survey where student attitudes are measured on a continuous scale:

- 0 = neutral,
- negative values = oppose,
- positive values = support.

We'll first run a random-sample survey of 50 students. Then we simulate many 50-student samples to see how much the number of opponents could vary. That variability informs the proposal planning.

8.1.1 Opinion Study

□ The GOAL: Generate 50 random numbers, compare each to zero, and count how many are below zero.

```
set.seed(1234) # For reproducibility  
# Generate 50 random numbers to represent student attitudes from survey  
# with a normal distribution (mean = 2, sd = 3)  
x <- rnorm(n = 50, mean = 2, sd = 3)  
print(x)
```

□ Comment (example): While the average opinion is supportive (mean = 2), some individuals express opposition (negative values). In other words, some values are below zero because the mean is 2, but the standard deviation is 3, allowing for negative values. That's normal sample-to-sample variation.

```
# Mark who opposes (score < 0), then count  
below_zero <- x < 0  
print(below_zero)  
  
# Count how many are below zero  
count <- sum(below_zero)  
print(count)
```

□ Comment (example): We generated 50 random numbers from a normal distribution with mean 2 and standard deviation 3. We then compared each number to zero, yielding a logical vector indicating which are below zero. Finally, we summed the logical vector to count the number of values below zero.

8.1.2 Simulation Over Many Samples

Now simulate many realistic samples to understand how the count of opponents might bounce around.

Repeat this process multiple times using `replicate()`. This time, let's generate 50 random numbers in 200 iterations.

□ The GOAL: Use `replicate` to repeat the counts across many simulated samples.

```
# □ Test  
values_below_zero <- replicate(  
  n = 200, # 200 hypothetical survey runs  
  expr = sum(rnorm(50, mean = 2, sd = 3) < 0) # each run: 50 students, count opponents)
```

Order of Operations:

1. `rnorm(n = 50, mean = 2, sd = 3)` generates a sample of 50 random numbers.

2. `sum(... < 0)` counts how many of those numbers are below zero.
3. `replicate(n = 200, expr = ...)` repeats this process 200 times.
4. The result is a vector of counts, one for each of the 200 samples.

```
# □ Investigate
# Useful summaries for discussion
mean(values_below_zero) # typical number of opponents
```

```
## [1] 12.77
```

```
sd(values_below_zero) # how much the count varies
```

```
## [1] 3.121831
```

```
quantile(values_below_zero, c(.1,.5,.9)) # 10th/50th/90th percentiles
```

```
## 10% 50% 90%
```

```
## 9 13 17
```

□ Comment (example): The mean indicates that, on average, the typical count of those opposed is about 13 across many samples of 50 students. The standard deviation shows that the count of opponents can vary by about 3.1 from sample to sample. The percentiles provide additional insights into the distribution of opponent counts, indicating that in 10% of the samples, there are 9 or fewer opponents, while in 90% of the samples, there are 17 or fewer opponents.

```
# Example planning thresholds (adjust to your context)
mean(values_below_zero >= 15) # How often do we see ≥15 opponents out of 50?
```

```
## [1] 0.28
```

If at least 15 opponents trigger a halt, the proposal's risk of being halted is about 28% based on our simulations.

```
# Alternative way to calculate probability of halt
# One-line simulation estimate:
risk_halt <- mean(replicate(200, sum(rnorm(50, 2, 3) < 0)) >= 15)
risk_halt
```

```
## [1] 0.2
```

```

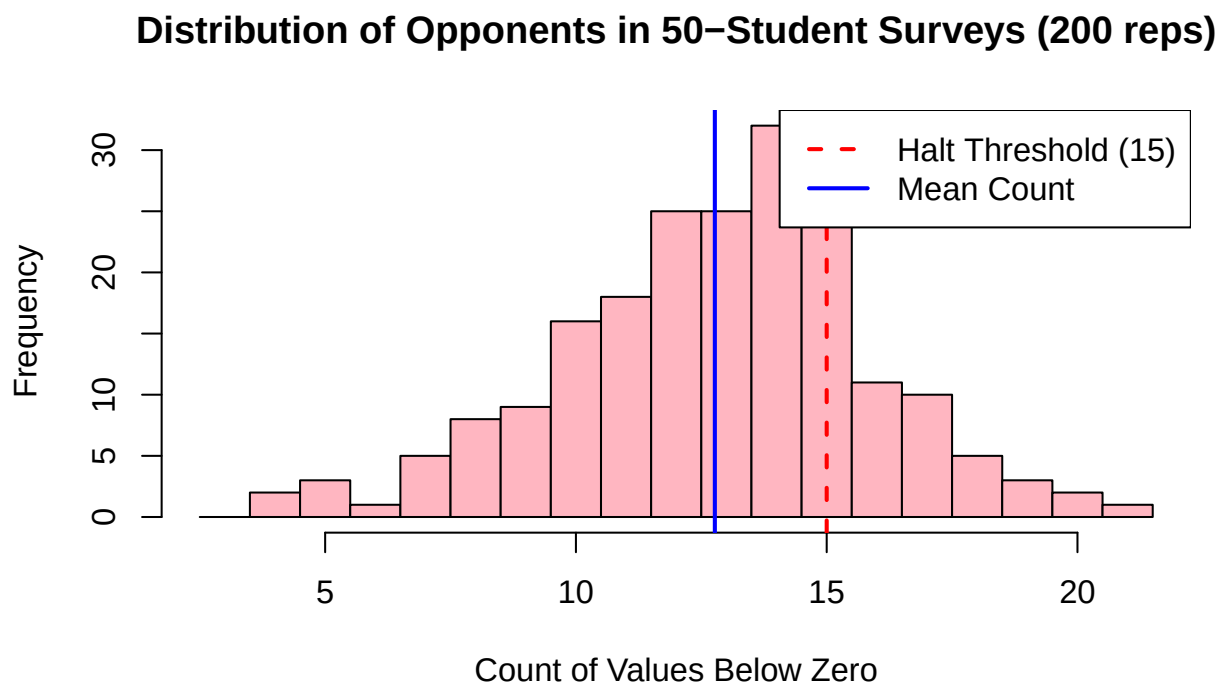
# Integer-aligned histogram + shading + on-plot probability
threshold <- 15
max_count <- max(values_below_zero)
breaks_int <- seq(2.5, max_count + 0.5, by = 1)

# Investigate
hist(values_below_zero,
     breaks = breaks_int,
     col = "lightpink",
     main = "Distribution of Opponents in 50-Student Surveys (200 reps)",
     xlab = "Count of Values Below Zero")

abline(v = threshold, col = "red", lwd = 2, lty = 2) # Halt threshold
abline(v = mean(values_below_zero), col = "blue", lwd = 2, lty = 1) # Mean line

legend("topright",
     legend = c("Halt Threshold (15)", "Mean Count"),
     col = c("red", "blue"),
     lty = c(2, 1),
     lwd = 2,
     bg = "white",
     bty = "o")

```



□ Comment (example): Each bar shows how often a particular count of opponents appeared across the 200 samples. The center of the histogram is the typical count. The spread shows how much that count changes from sample to sample, even if the underlying social climate doesn't change. About 1 in 4 samples hit the halt threshold (≥ 15).

9. ☒ Fair-Dice Check

Probability weights in `sample()` let you simulate biased outcomes. Recall the syntax:

- ☐ The SYNTAX: `sample(x, size, replace = FALSE, prob = NULL)`

9.1 Biased Die

The `prob` parameter specifies the probability weights for each element in the sampling pool.

- ☐ The GOAL: Simulate rolling a biased die 1,000 times with specified probabilities.

```
# ☐ Test  
rolls <- sample(1:6, 1000, prob = c(.12,.13,.07,.23,.22,.23), replace = TRUE)
```

- ☐ Comment (example): This simulates rolling one biased or “loaded” die 1000 times. The length of the `prob` vector must match the number of sampled values (`x = 1:6`). The weights usually sum to 1; R will normalize them if they don’t.

```
# ☐ Verify that relative frequencies should be near the weights  
round(prop.table(table(rolls)), 3)
```

```
## rolls  
##    1    2    3    4    5    6  
## 0.130 0.133 0.076 0.222 0.223 0.216
```

- ☐ Comment (example): The relative frequencies should roughly match the specified probabilities, confirming the bias in the die. The results may vary slightly due to randomness, but over many rolls, they should converge to the specified probabilities.

-
- ☐ Break Things! What happens if the length of `prob` does not match the length of `x`? Paste the error message and 1 sentence on why it occurs. What happens if the `prob` values do not sum to 1? Can you have negative probabilities? You could try `sample(1:6, 10, replace = TRUE, prob = c(.5, .5))` and try `sample(1:6, 10, replace = TRUE, prob = c(.1, .2, 0.2, .05, .2, .05, .4))` and `sample(1:6, 1000, prob = c(.22,-.13,.07,.23,.22,.23), replace = TRUE)`. Explain what happens and what the rules are for the `prob` argument in `sample()`. ____

9.2 Fair vs. Unfair Dice

□ Scenario. You are playing a dice game with a “friend” who claims their dice are fair. □ Your “friend’s” dice look suspiciously unfair. Compare it to our fair dice. We’ll test the your friend’s dice against your fair dice.

□ The GOAL: Simulate rolling two biased dice 1,000 times and compare to two fair dice. We’ll compare your friend’s results to fair dice by adding the the face values of the dice then

```
# □ Test
set.seed(8000)
biased <- c(.12, .13, .07, .23, .22, .23) # sums to 1
friend_sum <- sample(1:6, size = 1000, replace = TRUE, prob = biased) +
  sample(1:6, size = 1000, replace = TRUE, prob = biased)
```

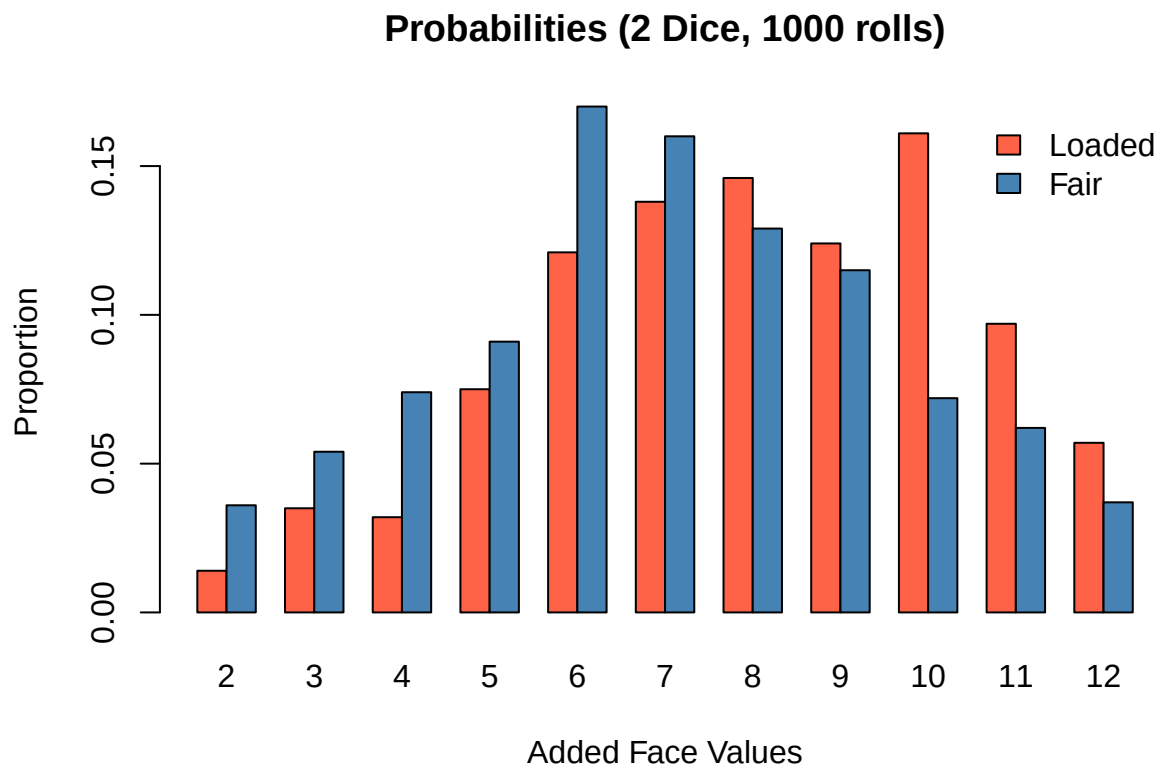
```
# □ Test
fair_sum <- sample(1:6, 1000, TRUE) + sample(1:6, 1000, TRUE) # no 'prob' = equal chance
```

□ Comment (example): The simulates rolling two biased dice 1000 times. This is like you and a friend each rolling two dice 1000 times simultaneously.

□ The GOAL: Compare the distributions of the sum of two fair dice face values from your friend’s biased dice.

```
# Proportions of each sum
ploaded <- prop.table(table(friend_sum))
pfair <- prop.table(table(fair_sum))
```

```
# □ Investigate
# Side-by-side bar plot
face_props <- rbind(Bias = ploaded, Fair = pfair)
barplot(face_props,
  beside = TRUE,
  col = c("tomato", "steelblue"),
  main = "Probabilities (2 Dice, 1000 rolls)",
  xlab = "Added Face Values",
  ylab = "Proportion")
legend("topright", legend = c("Loaded", "Fair"),
  fill = c("tomato", "steelblue"), bty = "n")
```

☐ Comment: What is the shape of the red bars compared with the blue?

10. ☒ Practice Space

For an *BONUS* 0.5 **technical skill** points, complete the practice below.

☐ Practice: Replace the blanks (____) and run.

☐ ♂☐☐ Your friend shows up to your DnD game with a big 3D printed 20-sided die and claims it is fair. You decide to test it. [How to Make Fair DnD Dice from ANY Shape](#)

☐ The GOAL: Simulate 300 rolls of a loaded or fair 20☐ sided die (values 1:20) and store in **big_die**

```
big_die <- sample(____, size = ____ , replace = ____, prob = ____)
```

```
# Using `any()`, did you roll a 20?
```

```
any(big_die == ____)
```

11. ☒ Assignment

☐ How far could a new fire run in 8 hours?

Today, you'll simulate the probability that an ignition's 8-hour straight-line spread distance (km) exceeds a threshold of 20 km.

Replace each ____ placeholder (and any TODO comments) with working code or a short written answer. Run each section; be sure the requested objects appear in the Environment. Use the template ☐ Goal → ☐ Code → ☐ Checks → ☐ Explain.

When finished, save **BOTH** you R code script file and your .RData workspace and upload to Canvas. ____

11.1 Dataset

Wildfire managers need to know where wildfires are likely to occur and how intense they might be. The U.S. Forest Service (USFS) Wildfire Hazard Potential (WHP) is a raster geospatial product [Dillon, G., Gilbertson-Day, J., 2020](#) paired with Idaho-only subset data provided by Dr. Matt Williamson, an Associate Professor at Boise State University in Human-Environment Systems. Dr. Williamson specializes in spatial analysis using R and teaches the graduate level R course HES 505 each fall. To learn more about Dr. Williamson work, visit his [SPASES Lab Page](#).

Figure 1 shows the raster that was processed in R using packages terra, sf, tidyverse, tidyterra, tigris. The GeoTIFF is projected in USA_Contiguous_Albers Equal Area Conic (USGS version) (Albers; +proj=aea +lon_0=-96 +lat_1=29.5 +lat_2=45.5 +lat_0=23; meters) with a cell size of 240×240 m. Within Idaho, valid raster cells cover 216,315 km² ($\approx$ the state area); cells outside the state footprint within the raster's bounding box are NoData ($\approx$ 231,477 km²). WHP values in the subset range from 0 to 64,185.656 (mean 1,615.668, SD 4,119.674). Distributional summaries are: P25 = 130.484, median = 397.297, P75 = 1,097.703, P90 = 3,507.469, P95 = 7,753.109, P99 = 22,429.154 (all unitless). WHP's native grid is 270 m; the Idaho file's 240 m cell size indicates the subset was resampled prior to delivery. Higher index values indicate greater relative hazard; the layer is continuous. TIGER/Line state cartographic boundary for Idaho (retrieved via the R tigris package and reprojected to the raster CRS).

Formal dataset citation

Dillon, Gregory K.; Gilbertson-Day, Julie W. 2020. Wildfire Hazard Potential for the United States (270-m), version 2020. 3rd Edition. Forest Service Research Data Archive. <https://doi.org/10.2737/RDS-2015-0047-3>.

Idaho subset (whp_id.tif) prepared and provided by Matt Williamson, SPASES Lab, Boise State University.

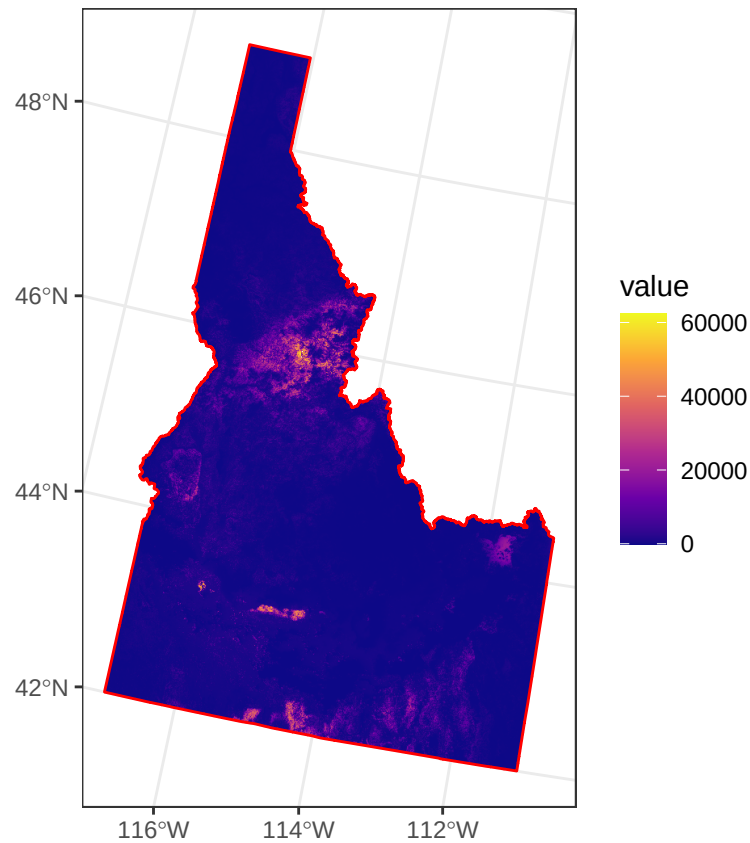


Figure 11.1: Figure 1. Idaho Wildfire Hazard Potential (WHP), continuous index (unitless) from the USFS WHP v2020 raster, clipped to the Idaho boundary. Colors use a viridis gradient (dark purple = lower WHP; yellow = higher WHP); values span approximately 0–64,186. Raster resolution is 240 m in an Albers Equal Area projection; NoData cells are transparent. Red outline shows the TIGER/Line state boundary reprojected to the raster CRS. Idaho subset prepared by Matt Williamson (SPASES Lab, Boise State University); original dataset: Dillon & Gilbertson-Day (2020), USFS Research Data Archive (DOI: 10.2737/RDS-2015-0047-3).

Table 1. summarizes the quantile-based wildfire hazard categories derived from Idaho’s continuous USFS WHP raster and the distance distributions used for an 8-hour run. Category lists seven bins (Very Low to Ultra) defined from the raster’s cumulative distribution; the WHP index itself is unitless. Probability is the statewide area share for each bin—expressed as a percent of valid raster area (NA excluded)—and is interpreted as the chance a random ignition falls in that category. Normal mean (km) and Normal SD (km) are the mean and standard deviation, in kilometers, for a zero-truncated normal distance model representing straight-line spread over 8 hours. Mean log and SD log are the parameters μ and σ of a lognormal distance model in which $\log(\text{distance in km}) \sim N(\mu, \sigma^2)$. Labels “Extreme” and “Ultra” denote upper-tail bins (approximately the 95–99% and 99–100% quantiles, respectively).

Parameters for this assignment were selected from the the following references. According to the Idaho Department of Environmental Quality (DEQ), the 2021 fire season saw 439 individual wildfires. A reasonable single-year ignition count for Idaho is ~ 450 (rounding the 2021 DEQ total of 439).

Category	Probability	Mean (km)	SD (km)	Mean log	SD log
Very Low	25	2.0	1.0	0.582	0.472
Low	25	3.0	1.0	1.046	0.325
Moderate	25	4.5	1.5	1.451	0.325
High	15	7.0	2.0	1.907	0.280
Very High	5	12.0	3.0	2.455	0.246
Extreme	4	18.0	4.0	2.866	0.220
Ultra (Top 1%)	1	28.0	6.0	3.310	0.212

11.2 Objective

Objective. Estimate short-term spread risk using simulation. Model each ignition’s 8-hour straight-line distance with a lognormal distribution. Do this two ways:

Statewide mixture: sample a WHP category using the Probability column in Table 1, then draw a distance from that category’s Mean log and SD log. An ignition critical if it distance ≥ 20 km

You will simulate 2,000 seasons, each with 450 ignitions. For each season, count how many ignitions are ≥ 10 km (major runs) and how many are critical under the rule. Summarize both expected counts and the probability the season exceeds a planning threshold (e.g., “ > 50 major runs” and “ > 10 critical runs”). Distance is used here as a proxy for severity; in practice agencies track area. Category probabilities approximate where ignitions occur and are a teaching simplification.

11.3 Lognormal Random Numbers

The lognormal model is used because most fires will move a modest distance, but a small fraction travel much farther. A lognormal model enforces non-negativite distance values and captures that right tail.

Use `rlnorm()` to generate lognormal random numbers.

```
?rlnorm # Check the help file for rlnorm()
```

□ The SYNTAX: `rlnorm(n, meanlog = 0, sdlog = 1)`

- `n` is the number of random numbers to generate. In this scenario, `n` is the number of ignitions to simulated per Monte Carlo run.
- `meanlog` is the mean of the logarithm of the distribution with default set to 0.
- `sdlog` is the standard deviation of the logarithm of the distribution with default set to 1.

Note: You should always use the log-scale parameters of mean and SD in `rlnorm()`.

11.4 Task 1

□ Goal: Start by setting a seed. Generate a large vector of simulated 8-hour distances for the **High** category using `rlnorm()`. Look up the **High** row's **Mean log** and **SD log** in **Table 1**, and store the result as `High_Lognorm`. Set the number of ignitions per season to 450.

```
set.seed(12345)
# □ Test
High_Lognorm <- rlnorm(n = 450, meanlog = 1.907, sdlog = 0.280)
```

11.5 Task 2

□ / □ Checks and Comment

Summarize what you just simulated. Report the median, mean, and 90th percentile of `High_Lognorm`, and comment in one sentence on the right-skew you expect for a lognormal.

```
# □ Verify
median(High_Lognorm)
```

```
## [1] 7.035328
```

```
mean(High_Lognorm)
```

```
## [1] 7.224904
```

```
quantile(High_Lognorm, 0.9)
```

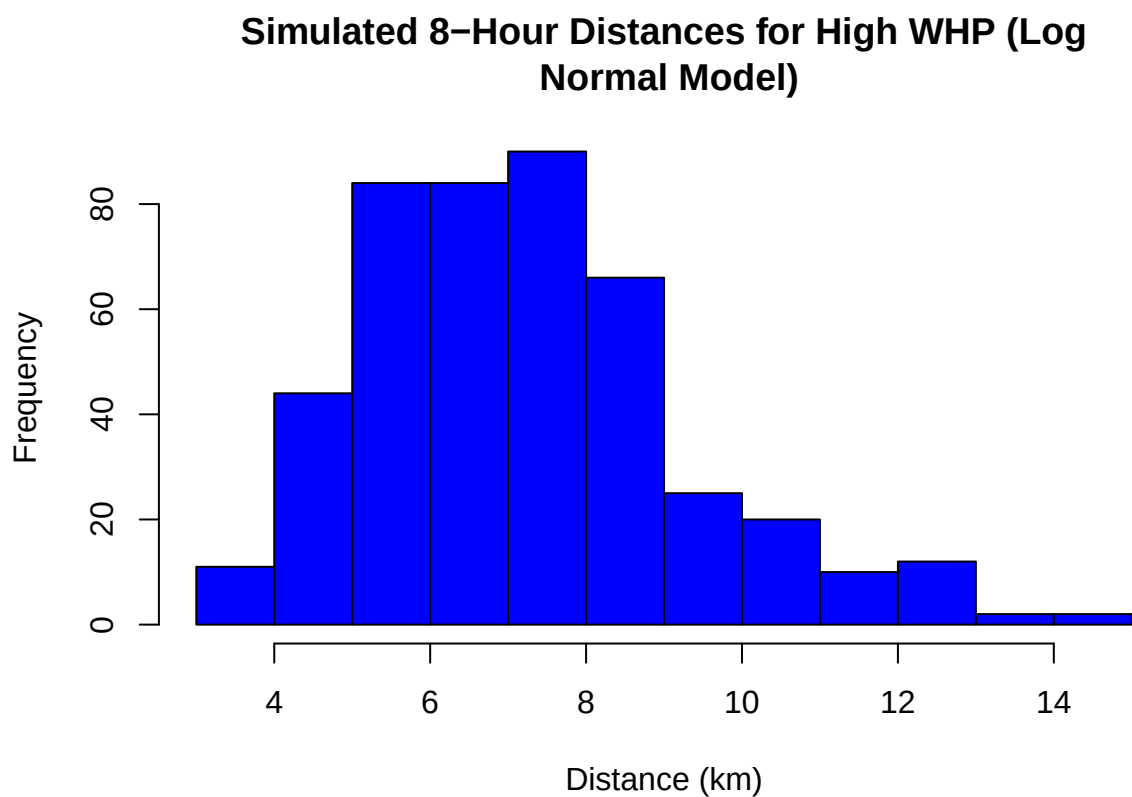
```
## 90%
## 10.03975
```

11.6 Task 3

□ / □ Checks and Comment

Make a histogram of High_Lognorm. In one sentence, note how the shape matches your summary in Task 2.

```
# □ Verify
hist(High_Lognorm,
     col = "blue",
     main = "Simulated 8-Hour Distances for High WHP (Log
Normal Model)",
     xlab = "Distance (km)")
```



11.7 Task 4

□ Goal: Copy the code that created High_Lognorm and nest it inside of the mean() function. Set the rlnorm() call to >= 10. This one line will compute the proportion of a single sample that reaches or exceeds a threshold of 10 km spread in 8-hours. Store this result as p_high_one

```
# □ Test
p_high_one <- mean(rlnorm(n = 450, meanlog = 1.907, sdlog = 0.280) >= 10) # one season, 450 ignitions
```

11.8 Task 5

□ / □ Checks and Comment

Comment about meaning of `p_high_one`. What does this value represent in the context of wildfire spread risk for ignitions in the **High** WHP category?

```
# □ Verify  
p_high_one
```

```
## [1] 0.07333333
```

11.9 Task 6

□ Goal: Turn that single-season calculation into many seasons. Use `replicate()` to repeat the same inside expression for 2,000 seasons. Store the resulting vector of proportions as `p_high_many`, then report its mean and make a small histogram of those season-level proportions.

```
# □ Test  
p_high_many <- replicate(  
  n = 2000, # 2,000 seasons  
  expr = mean(rlnorm(n = 450, meanlog = 1.907, sdlog = 0.280) >= 10) # 450 wildfires, 10 km threshold  
)
```

```
# □ Verify  
mean(p_high_many)
```

```
## [1] 0.07926556
```

```
mean(p_high_many >= 1)
```

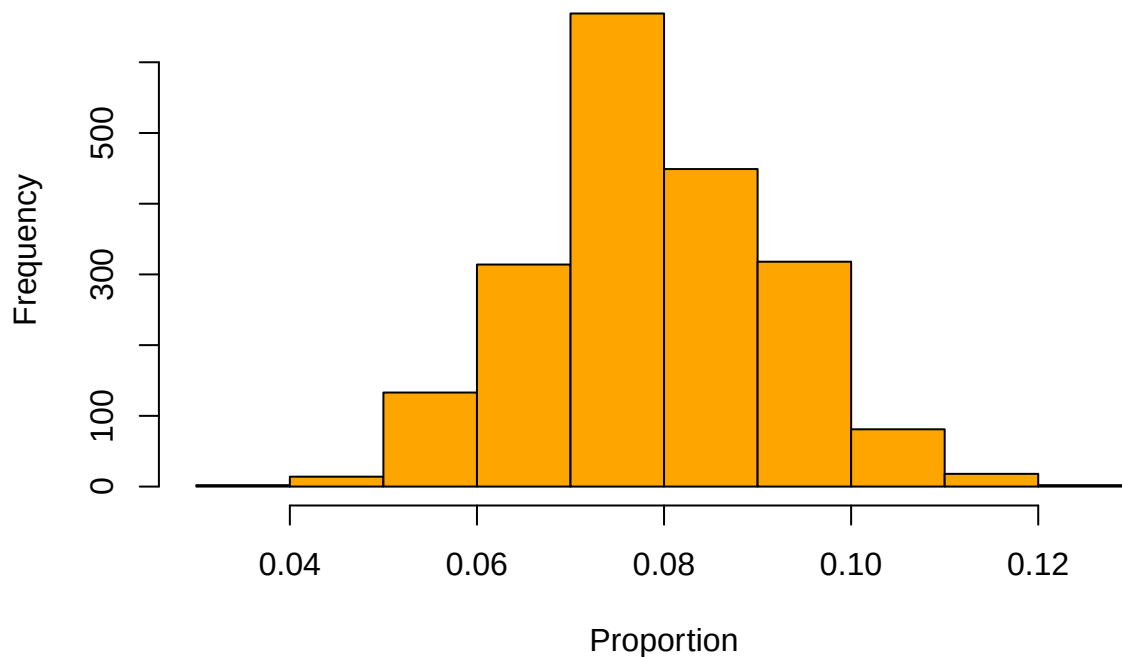
```
## [1] 0
```

11.10 Task 7

□ / □ Checks and Comment

```
# □ Verify  
hist(p_high_many,  
  col = "orange",  
  main = "Season-level proportions  $\geq k$  (High)",  
  xlab = "Proportion")
```


Season-level proportions = k (High)



11.11 Task 8

□ Goal: Create four parallel vectors from Table 1: cats, prob (as proportions that sum to 1), meanlog, and sdlog in matching order. Build the inside expression for a single season of $N <- 450$ ignitions: first sample a category index for each ignition using `sample()` with prob, then draw distances with `rlnorm()` using the matched meanlog and sdlog. Return a count of ignitions $\geq k$. Run it once to confirm it works.

```
# □ Test
cats <- c("Very Low", "Low", "Moderate", "High", "Very High", "Extreme", "Ultra")
prob <- c(0.25, 0.25, 0.25, 0.15, 0.05, 0.04, 0.01) # proportions sum to 1
meanlog <- c(0.582, 1.046, 1.451, 1.907, 2.455, 2.866, 3.310)
sdlog <- c(0.472, 0.325, 0.325, 0.280, 0.246, 0.220, 0.212)
N <- 450 # ignitions per season
one_mixture_count <- {
  idx <- sample.int(length(cats), size = N, replace = TRUE, prob = prob)
  d <- rlnorm(N, meanlog = meanlog[idx], sdlog = sdlog[idx])
  sum(d >= 10)
}
one_mixture_count
```

```
## [1] 34
```

11.12 Task 9

□ Goal: Wrap the mixture inside code from Task 7 in `replicate()` for S seasons. Save the season counts as `counts_mix`. Report the expected number of $\geq k$ runs and the probability that a season has more than 10 such runs (change the threshold if instructed).

```
# □ Test
counts_mix <- replicate(
  n = 2000,                                # 2,000 seasons
  expr = {
    idx <- sample.int(length(cats), size = N, replace = TRUE, prob = prob)
    d <- rlnorm(N, meanlog = meanlog[idx], sdlog = sdlog[idx])
    sum(d >= 10)
  }
)
```

11.13 Task 10

□ / □ Checks and Comment

```
# □ Verify
mean(counts_mix)      # expected number of  $\geq k$  runs
```

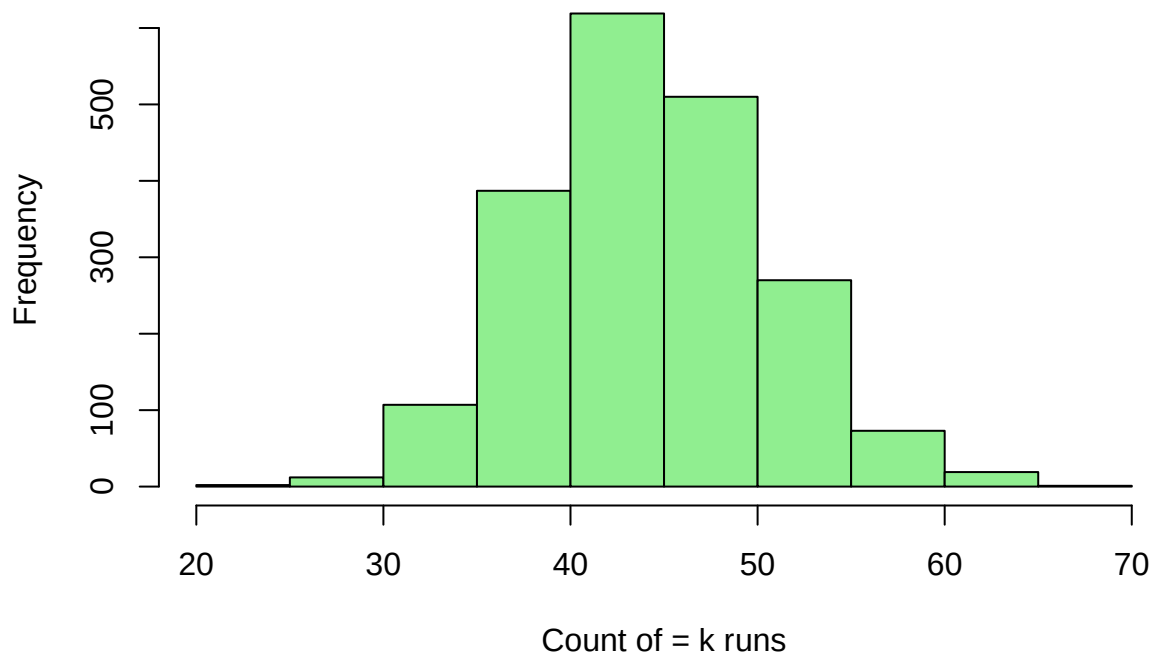
```
## [1] 44.7605
```

```
mean(counts_mix > 10) # probability of > 10 such runs
```

```
## [1] 1
```

```
# □ Verify
hist(counts_mix,
  col = "lightgreen",
  main = "Season-level counts of  $\geq k$  runs (Mixture Model)",
  xlab = "Count of  $\geq k$  runs")
```

Season-level counts of = k runs (Mixture Model)



11.14 Reflections

□ Comment: In 1-2 sentences, reflect on what you learned from this assignment about using simulation to estimate wildfire spread risk. Mention any challenges you faced and how you overcame them.

12. Save and Upload

1. You will be submitting **both** the *Your Pick* Document and the workspace file. The workspace file saves all the objects in your environment that you created in this lesson. You can save the workspace by running the following command in a code chunk of the Quarto Document document:

```
save.image("Assignment12_Workspace.RData")
```

Or you can click the “Save Workspace” button in the Environment pane.

□ **Always save the R documents before closing.**

2. Find the assignment in this week’s module in Canvas and upload **both** the RMD and the workspace file.

13. Today you practiced:

- Used `sample()` for discrete simulations with or without replacement.
- Visualized uniform and aggregated dice results with histograms.
- Generated normal random numbers with `rnorm()` and plotted the shape.
- Repeated an experiment easily with `replicate()`.
- Compared two simulated distributions to spot bias.